论文全文翻译 & 逐段精读解读

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention

Katharopoulos, Vyas, Pappas, Fleuret (ICML 2020)

阅读指南

本文档按照原论文顺序，提供**逐段全文翻译**与**超详细解读**。文档采用以下颜色编码：

- 灰色框 = **原文完整翻译**（忠实翻译原论文每一句话）
- 绿色框 = 通俗解释（用大白话说明这段在讲什么）
- 蓝色框 = 数学公式的超详细推导
- 橙色框 = 关键概念/总结/重要结论
- 红色框 = 需要特别注意的地方
- 【引用说明：紫色标注】 = 解释每个引用文献的来源与被引用的原因

所有数学符号都会在第一次出现时给出详细解释。推导过程力求让刚上大学的同学也能看懂。
文末附有**完整参考文献一览表**（附录 B），汇总了全部 39 篇引用文献的出处与引用原因。

Contents

1 摘要 (Abstract) **3**

2 Introduction（引言） **3**

2.1 第一段：Transformer 的成功 3

2.2 第二段：Transformer 的瓶颈 4
2.3 第三段：前人的改进方法 5
2.4 第四段：本文的贡献 5

3 Related Work（相关工作） 6
3.1 高效 Transformer（Efficient Transformers） 6
3.2 理解自注意力（Understanding Self-Attention） 7
3.3 线性化 Softmax（Linearized Softmax） 8

4 Linear Transformers（线性 Transformer）——核心方法 8
4.1 标准 Transformer 的数学描述 9
4.1.1 自注意力的详细计算 9
4.1.2 广义注意力公式 12
4.2 Linearized Attention（线性化注意力）——本文最核心的部分 13
4.2.1 特征映射函数的选择 16
4.3 Causal Masking（因果掩码） 17
4.3.1 梯度计算 19
4.3.2 训练与推理 21
4.4 Transformers are RNNs（Transformer 就是 RNN！） 21

5 Experiments（实验） 24
5.1 合成任务 24
5.1.1 收敛性分析 24
5.1.2 计算和内存需求 25
5.2 图像生成 26
5.2.1 MNIST 26
5.2.2 CIFAR-10 28
5.3 语音识别 29
5.4 训练演化对比（补充材料） 31
5.5 补充材料：图像生成吞吐量讨论 31

6 Conclusions（结论） 32

A 附录 A：线性因果注意力的前向和反向传播伪代码 34

B 附录 B：完整参考文献一览表 35

1 摘要 (Abstract)

原文翻译

Transformer 在多种任务上取得了卓越的性能，但由于其计算复杂度相对于输入长度呈二次方关系，对于非常长的序列而言速度极慢。为了解决这一限制，我们将自注意力表示为核特征映射（kernel feature maps）的线性点积，并利用矩阵乘积的结合律（associativity property），将复杂度从 $\mathcal{O}(N^2)$ 降低至 $\mathcal{O}(N)$，其中 N 为序列长度。我们表明，这一表述允许一种迭代式的实现方式，能够极大地加速自回归 Transformer，并揭示了它们与循环神经网络（RNN）之间的关系。我们提出的**线性 Transformer**（linear transformers）达到了与标准 Transformer 相似的性能，并且在超长序列的自回归预测上速度提升最高可达 **4000 倍**。

大白话解释摘要

问题： Transformer 很强，但太慢了。处理长度为 N 的序列时，计算量是 N^2 级别的。如果 $N = 1000$，就要算 100 万次；$N = 10000$ 就要算 1 亿次！
方法： 用一个数学技巧（核函数 + 矩阵乘法的结合律），改变了注意力的计算顺序，使得计算量变成 N 级别的——线性增长！
额外发现： 改造后的 Transformer 居然可以写成 RNN 的形式！这意味着在自回归推理时，每预测一个新元素只需固定的计算量（不随序列变长而增加）。
效果： 性能差不多，速度快了最多 4000 倍。

2 Introduction（引言）

2.1 第一段：Transformer 的成功

原文翻译

Transformer 模型最初由 Vaswani 等人【引用说明：即”Attention is All You Need”（2017），首次提出了 Transformer 架构，是本文所改进的基础模型】在神经机器翻译【引用说明：Sutskever 等人（2014）提出了基于编码器-解码器的序列到序列学习框架，是神经机器翻译的奠基性工作；Bahdanau 等人（2015）在此基础上引入了注意力机制，使模型能够在解码时动态关注输入的不同部分】的背景下提出，并已在处理自然语言【引用说明：Devlin 等人（2019）提出了 BERT，通过双向预训练在多个 NLP 任务上取得突破，展示了 Transformer 在自然语言领域的强大能力】、音频【引用说明：Sperber 等人（2018）将自注意力模型应用于语音任务，展示了 Transformer 在音频领域的有效性】和图像【引用说明：Parmar 等人（2019）提出了独立的自注意力视觉模型，证明了 Transformer 架构在图像任务上的可行性】的多种任务上展现出令人印象深刻的结果。除了有充足监督信号的任务之外，Transformer 在通过自回归目标【引用说明：Radford 等人（2018）即 GPT-1，提出了用自回归预训练的方式学习语言表示；Radford 等人（2019）即 GPT-2，进一步展示了大规模语言模型的无监督多任务学习能力】或掩码语言建模目标【引用说明：Devlin 等人（2019）即 BERT，通过掩码语言建模预训练；Yang 等人（2019）即 XLNet，提出了广义自回归预训练方法；Song 等人（2019）即 MASS，提出了掩码序列到序列预训练；Liu 等人（2020）即 RoBERTa，对 BERT 预训练策略进行了鲁棒性优化】进行预训练后，也能有效地将知识迁移到监督信号有限或没有监督信号的任务上。

大白话解释

这段在说 Transformer 有多厉害：

- 它最初是为翻译设计的（比如中文翻英文）
- 后来发现它在几乎所有领域都好用——文字、声音、图片
- GPT 系列（自回归预训练）和 BERT 系列（掩码语言建模预训练）都是 Transformer

关键术语：

- **自回归 (Autoregressive)**：依次生成序列中的每个元素。比如写文章时，一个字一个字地写，每写一个新字都要参考前面写过的所有字。
- **掩码语言建模 (Masked LM)**：把句子中的一些词遮住，让模型猜被遮住的词是什么。这就是 BERT 的训练方式。

2.2 第二段：Transformer 的瓶颈

原文翻译

然而，这些好处往往伴随着非常高的计算和内存代价。瓶颈主要由自注意力的全局感受野造成——处理 N 个输入的上下文需要 $\mathcal{O}(N^2)$ 的内存和时间复杂度。因此，在实践中 Transformer 训练缓慢，且其上下文是受限的。这破坏了时间一致性（temporal coherence），并妨碍了捕获长程依赖关系。Dai 等人【引用说明：即 Transformer-XL（2019），通过跨段递归机制使模型能够利用更长的上下文，在语言建模上取得了当时的最优结果。这里引用是为了说明：已有人尝试解决长程依赖问题，但代价是增加了计算量】通过关注来自先前上下文的记忆来解决后者的问题，但代价是牺牲了计算效率。

大白话解释

这段指出了 Transformer 最大的问题：**它太慢了！**
为什么慢？因为自注意力要让序列中的**每个**位置都和**所有其他**位置做比较。如果有 N 个位置，那就是 $N \times N = N^2$ 次比较。
具体例子：

- 一张 28×28 的 MNIST 灰度图：$N = 784$，$N^2 \approx 61$ 万
- 一张 $32 \times 32 \times 3$ 的 CIFAR 彩色图：$N = 3072$，$N^2 \approx 943$ 万
- 一篇 4000 词的文章：$N = 4000$，$N^2 = 1600$ 万

这就导致了两个后果：(1) 训练很慢，(2) 不能处理太长的序列（内存放不下那个 $N \times N$ 的注意力矩阵）。

2.3 第三段：前人的改进方法

原文翻译

近年来，研究者们将注意力转向了在不牺牲效率的情况下增加上下文长度的方法。为此，Child 等人【引用说明：即 Sparse Transformer（2019），对注意力矩阵进行稀疏分解，让每个位置只关注部分其他位置，而非全部，从而降低计算量。这里引用是作为本文之前的效率改进基线之一】引入了注意力矩阵的稀疏分解，将自注意力的复杂度降低到 $\mathcal{O}\left(N\sqrt{N}\right)$。Kitaev 等人【引用说明：即 Reformer（2020），使用局部敏感哈希（LSH）将相似的 query 和 key 分组，只在组内计算注意力。这是本文在实验中的主要对比基线之一】利用局部敏感哈希（locality-sensitive hashing）进一步将复杂度降低到 $\mathcal{O}(N \log N)$。这使得扩展到长序列成为可能。尽管上述模型可以在长序列上进行高效训练，但**它们并没有加速自回归推理**。

大白话解释

在本文之前，已经有人尝试解决这个问题：
Sparse Transformer（2019）：不让每个位置和所有位置比较，只和部分位置比较（稀疏注意力）。复杂度 $N^2 \to N\sqrt{N}$。
Reformer（2020）：用”局部敏感哈希”(LSH) 技术，只在相似的位置之间计算注意力。复杂度 $N^2 \to N \log N$。
但有个关键问题：这些方法只加速了**训练**（因为训练时整个序列已知，可以预先决定哪些位置互相比较），而在**推理**（自回归生成）时，由于要一步步生成，这些优化无法充分发挥。
本文的方法不仅加速训练，还能**极大加速推理**——这是最大的区别。

2.4 第四段：本文的贡献

原文翻译

在本文中，我们引入了线性 Transformer 模型，它显著减少了内存占用，且相对于上下文长度呈线性扩展。我们通过使用基于核函数的自注意力表述以及矩阵乘积的结合律来计算自注意力权重（§ 3.2）以实现这一目标。利用我们的线性化表述，我们还以线性复杂度和常数内存表达了因果掩码（§ 3.3）。这揭示了 Transformer 与 RNN 之间的关系，使我们能够以快几个数量级的速度执行自回归推理（§ 3.4）。我们在图像生成和自动语音识别上的评估表明，线性 Transformer 能够达到标准 Transformer 的性能水平，同时在推理时快了最多三个数量级。

本文三大贡献

1. **线性注意力**（§ 3.2）：复杂度从 $\mathcal{O}(N^2)$ 降到 $\mathcal{O}(N)$
2. **因果掩码也能线性**（§ 3.3）：自回归训练和推理都是线性复杂度
3. **揭示 Transformer = RNN**（§ 3.4）：推理时可以像 RNN 一样逐步计算，每步只需常数时间

3 Related Work（相关工作）

原文翻译——章节引言

在本节中，我们概述了旨在解决 Transformer 大量内存和计算需求的最相关工作。此外，我们讨论了从理论上分析 Transformer 核心组件——即自注意力——的方法。最后，我们介绍了另一条寻求缓解注意力计算中 softmax 瓶颈的研究路线。

3.1 高效 Transformer（Efficient Transformers）

原文翻译

现有工作通过权重剪枝【引用说明：Michel 等人（2019），研究了多头注意力中各头的重要性，发现很多头可以被移除而不显著影响性能。这里引用是作为" 通过减少参数来提高效率" 这一思路的代表】、权重分解【引用说明：Lan 等人（2020）即 ALBERT，通过跨层参数共享和嵌入矩阵分解大幅减少参数量。引用原因同上】、权重量化【引用说明：Zafrir 等人（2019）即 Q8BERT，将 BERT 量化为 8 位整数进行推理。引用原因同上】或知识蒸馏来提高 Transformer 的内存效率。Clark 等人【引用说明：即 ELECTRA（2020），提出了" 替换词检测"(replaced token detection) 这一新型预训练目标，比掩码语言建模更高效。引用它是为了展示" 另一种减少计算的思路"】提出了一种名为" 替换词检测" 的新型预训练目标，这种方法在样本效率上更高，从而减少了整体计算量。Lample 等人【引用说明：即 Product-Key Attention（2019），提出用乘积键注意力来增加层的容量而几乎不增加计算开销。引用它是为了展示在不改变渐近复杂度的情况下改善效率的方法】使用了乘积键注意力，以几乎可忽略的计算开销增加了任意层的容量。用这些方法减少内存或计算需求确实能带来训练或推理时间的加速，但从根本上说，时间复杂度对于序列长度仍然是二次方的，这阻碍了扩展到长序列。相比之下，我们表明我们的方法在理论上（§ 3.2）和实证上（§ 4.1）都降低了 Transformer 的内存和时间复杂度。

原文翻译——上下文扩展

另一条研究路线旨在增加 Transformer 中自注意力的" 上下文"。上下文指的是用于计算自注意力的序列的最大部分。Dai 等人【引用说明：再次引用 Transformer-XL，在这里是作为" 增加上下文长度" 这条路线的代表作】引入了 Transformer-XL，它通过学习超越固定长度上下文的依赖关系而不破坏时间一致性，在语言建模上达到了最先进的水平。然而，在内存中保留先前的上下文引入了显著的额外计算成本。相比之下，Sukhbaatar 等人【引用说明：即 Adaptive Attention Span（2019），让每个注意力头自动学习其最优的注意力跨度（即关注多远的历史），从而在保持效果的同时控制计算量。引用它是为了与本文形成对比——它们渐近复杂度没变，而本文改善了渐近复杂度】通过为每个注意力头学习最优的注意力跨度，在保持对内存占用和计算时间的控制的同时，显著扩展了上下文长度。注意，这两种方法的渐近复杂度与标准模型相同。相比之下，我们改善了自注意力的渐近复杂度，从而允许我们使用显著更大的上下文。

原文翻译——与 Sparse Transformer 和 Reformer 的比较

与我们模型更相关的是 Child 等人【引用说明：Sparse Transformer，此处再次引用以进行详细对比】和 Kitaev 等人【引用说明：Reformer，此处再次引用以进行详细对比】的工作。前者引入了注意力矩阵的稀疏分解，将生成长序列的整体复杂度从二次方降低到 $\mathcal{O}\left(N\sqrt{N}\right)$。更近期的 Kitaev 等人提出了 Reformer。该方法通过使用局部敏感哈希（LSH）减少点积运算数量，将复杂度进一步降低到 $\mathcal{O}(N \log N)$。注意，为了能够使用 LSH，Reformer 将注意力中的键（Key）限制为必须与查询（Query）相同。因此，该方法无法用于解码任务——在这些任务中键需要不同于查询。相比之下，线性 Transformer 对查询和键不施加任何约束，且相对于序列长度呈线性扩展。此外，它们可以用于在自回归任务中执行快三个数量级的推理，同时在验证困惑度方面达到可比的性能。

大白话解释——相关工作对比

本文和之前方法的核心区别：
vs 高效 Transformer（剪枝/量化等）：那些方法减少了常数因子，但 N^2 的根本问题没变。本文**把 N^2 变成了 N**。
vs Transformer-XL / Adaptive Span：它们扩展了上下文范围，但渐近复杂度没变。本文**改善了渐近复杂度**。
vs Sparse Transformer：$N^2 \to N\sqrt{N}$，但不加速推理。
vs Reformer：$N^2 \to N \log N$，但要求 $K = Q$，在解码任务中不能用。本文没有这个限制，且推理快了三个数量级。

3.2 理解自注意力（Understanding Self-Attention）

原文翻译

从理论角度更好地理解自注意力方面已有少数努力。Tsai 等人【引用说明：即”Transformer Dissection”（2019），提出了从核函数的视角来理解注意力，将注意力看作是一种核平滑器（kernel smoother），核的分数就是输入之间的相似度。本文的线性化思路正是受此启发——但 Tsai 等人没有用这个视角来加速计算，而本文做到了】提出了一种基于核函数的 Transformer 注意力表述，将注意力看作在输入上应用一个核平滑器（kernel smoother），核分数就是输入之间的相似度。这一表述提供了更好地理解注意力组件和整合位置编码的方式。相比之下，我们使用核函数表述来加速自注意力的计算并降低其计算复杂度。此外，我们观察到，如果将一个具有正相似度分数的核应用于查询和键，线性注意力能够正常收敛。
更近期的，Cordonnier 等人【引用说明：即”On the Relationship Between Self-Attention and Convolutional Layers”（2020），从理论上证明了具有足够头数的多头自注意力可以表达任何卷积层。引用它是为了展示”理解自注意力”这个研究方向的另一个角度——本文提供的是”Transformer = RNN”这一新视角】提供了理论证明和经验证据，表明具有足够数量头的多头自注意力可以表达任何卷积层。而我们展示的是：用自回归目标训练的自注意力层可以被看作一个循环神经网络，并且这一观察可以用于显著加速自回归 Transformer 模型的推理时间。

3.3 线性化 Softmax（Linearized Softmax）

原文翻译

多年来，softmax 一直是训练具有大量类别的分类模型的瓶颈【引用说明：Goodman（2001）提出基于类别的分层 softmax；Morin & Bengio（2005）提出层次化 softmax；Mnih & Hinton（2009）提出可扩展的层次化语言模型。这三篇引用共同说明：”softmax 的计算代价” 是一个长期存在的经典问题】。近期的工作【引用说明：Blanc & Rendle（2017）提出自适应采样 softmax；Rawat 等人（2019）提出基于随机傅里叶特征的采样 softmax。引用它们是因为它们用” 特征映射的线性点积来近似 softmax” 的思路直接启发了本文的核心方法】用特征映射的线性点积来近似 softmax，通过采样来加速训练。受这些工作启发，我们对 Transformer 中的 softmax 注意力进行了线性化。与本工作同期，Shen 等人【引用说明：即”Efficient Attention”（2020），也探索了线性化注意力，但只用于图像目标检测任务。引用它是为了表明线性化注意力的想法不是本文独创——但本文的贡献在于将其扩展到自回归模型并揭示了与 RNN 的联系】探索了将线性化注意力用于图像目标检测任务。相比之下，我们不仅线性化了注意力计算，还开发了一种在推理和训练中都具有线性复杂度和常数内存的自回归 Transformer 模型。此外，我们通过核函数的视角表明，每一个 Transformer 都可以被看作一个循环神经网络。

大白话解释

vs Tsai 等人（核函数视角）：他们提出了” 注意力可以看作核函数” 的理论，但没有用它来加速计算。本文**用核函数来实际加速**。
vs Cordonnier 等人（注意力 = 卷积）：他们揭示了自注意力与卷积的关系。本文揭示的是**自注意力与 RNN 的关系**。
vs Shen 等人（Efficient Attention）：他们也在同期线性化了注意力，但只用于一个特定任务。本文不仅线性化，还**发展了完整的自回归模型**，并揭示了 Transformer 与 RNN 的关系。
vs 线性化 softmax 的采样方法：那些方法启发了本文用” 特征映射的线性点积” 来替代 softmax 的核心思路。

4 Linear Transformers（线性 Transformer）——核心方法

原文翻译

在本节中，我们正式提出我们的线性 Transformer。我们展示，将注意力从传统的 softmax 注意力改为基于特征映射的点积注意力，不仅能获得更好的时间和内存复杂度，还能得到一个可以在线性时间内执行序列生成的因果模型，类似于循环神经网络。
首先，在 § 3.1 中，我们介绍 Vaswani 等人【引用说明：再次引用原始 Transformer 论文，因为后续推导需要从标准 Transformer 的定义出发】引入的 Transformer 架构的一种表述。随后，在 § 3.2 和 § 3.3 中，我们提出我们的线性 Transformer。最后，在 § 3.4 中，我们将 Transformer 改写为循环神经网络。

4.1 标准 Transformer 的数学描述

原文翻译

令 $x \in \mathbb{R}^{N\times F}$ 表示一个由 N 个 F 维特征向量组成的序列。Transformer 是一个函数 $T : \mathbb{R}^{N\times F} \to \mathbb{R}^{N\times F}$，由 L 个 Transformer 层 $T_1(\cdot), \ldots, T_L(\cdot)$ 依次复合而成。每一层定义如下：

$$T_l(x) = f_l(A_l(x) + x) \tag{1}$$

函数 $f_l(\cdot)$ 独立地变换每个特征，通常用一个小的两层前馈网络实现。$A_l(\cdot)$ 是自注意力函数，是 Transformer 中唯一跨序列位置交互的部分。

公式 (1) 超详细解读

符号含义：

- $x \in \mathbb{R}^{N\times F}$：输入序列，是一个 N 行 F 列的矩阵
  - N = 序列长度（比如句子有多少个词，图片有多少个像素）
  - F = 每个位置的特征维度（比如每个词用一个 256 维的向量表示）
  - 第 i 行 x_i 就是第 i 个位置的特征向量
- $A_l(\cdot)$: 第 l 层的**自注意力函数**——这是 Transformer 的灵魂, 让每个位置能”看到” 其他所有位置
- $+ x$：**残差连接 (residual connection)**——把输入直接加到输出上
- $f_l(\cdot)$：**前馈网络 (FFN)**——一个小的两层全连接网络，对每个位置独立处理

直觉理解： 每一层做的事情就是：

输出 = 前馈网络（自注意力的结果 + 原始输入）

残差连接的作用：防止网络太深导致梯度消失。相当于” 即使 A_l 没学到什么，至少把原来的 x 传过去”。

4.1.1 自注意力的详细计算

原文翻译

自注意力函数 $A_l(\cdot)$ 对每个位置计算所有其他位置的特征表示的加权平均，权重与表示之间的相似度分数成正比。形式化地, 输入序列 x 分别被三个矩阵 $W_Q \in \mathbb{R}^{F\times D}$、$W_K \in \mathbb{R}^{F\times D}$ 和 $W_V \in \mathbb{R}^{F\times M}$ 投影为对应的表示 Q、K 和 V。所有位置的输出 $A_l(x) = V'$ 按如下方式计算：

$$\begin{aligned} Q &= xW_Q, \\ K &= xW_K, \\ V &= xW_V, \\ A_l(x) = V' &= \mathrm{softmax}\left(\frac{QK^T}{\sqrt{D}}\right) V \end{aligned} \tag{2}$$

注意在上面的等式中，softmax 函数是按行（rowwise）作用于 QK^T 的。按照通

用术语，Q、K 和 V 分别被称为”查询”（queries）、”键”（keys）和”值”（values）。

公式 (2) 超详细逐步推导

第一步：投影——生成 Q、K、V

什么是”投影”？就是用矩阵乘法把原始向量变换到新的空间中。

$$Q = xW_Q \quad \text{大小：} (N \times F) \times (F \times D) = N \times D \tag{3}$$
$$K = xW_K \quad \text{大小：} (N \times F) \times (F \times D) = N \times D \tag{4}$$
$$V = xW_V \quad \text{大小：} (N \times F) \times (F \times M) = N \times M \tag{5}$$

符号含义表：

符号	大小	含义
x	$N \times F$	输入序列
W_Q	$F \times D$	Query 投影矩阵（可学习的参数）
W_K	$F \times D$	Key 投影矩阵（可学习的参数）
W_V	$F \times M$	Value 投影矩阵（可学习的参数）
Q	$N \times D$	查询矩阵（”我想找什么”）
K	$N \times D$	键矩阵（”我有什么可以被找到的”）
V	$N \times M$	值矩阵（”我实际携带的信息”）
D	标量	Query 和 Key 的维度
M	标量	Value 的维度

直觉理解——用图书馆来类比：

- Q（Query/查询）= 你去图书馆，手上拿着一张”搜索清单”
- K（Key/键）= 每本书封面上的”关键词标签”
- V（Value/值）= 每本书里面的”实际内容”
- 注意力机制 = 拿你的搜索清单去和每本书的标签比对，越相关的书你读得越仔细

第二步：计算注意力矩阵 QK^T

QK^T 的大小：$(N \times D) \times (D \times N) = N \times N$
第 (i, j) 个元素是第 i 个 Query 和第 j 个 Key 的**点积**：

$$(QK^T)_{ij} = Q_i \cdot K_j = \sum_{d=1}^{D} Q_{id} \cdot K_{jd}$$

具体的小例子：假设 $N = 3$，$D = 2$：

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

计算 QK^T（逐元素展示）：

$$QK^T = \begin{pmatrix} 1\times1+0\times1 & 1\times0+0\times1 & 1\times1+0\times0 \\ 0\times1+1\times1 & 0\times0+1\times1 & 0\times1+1\times0 \\ 1\times1+1\times1 & 1\times0+1\times1 & 1\times1+1\times0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & 1 \end{pmatrix}$$

这个 3×3 矩阵里，每个数字代表对应 Query 和 Key 之间的”原始相似度”。

第三步：缩放——除以 $\sqrt{D}$

$$\frac{QK^T}{\sqrt{D}}$$

为什么要除以 $\sqrt{D}$? 因为 D 个数字相加，结果的方差大约是 D 倍。除以 $\sqrt{D}$ 可以让数值重新稳定在一个合适的范围。
如果不除：点积值可能非常大 →softmax 之后几乎全部集中在最大值上 → 梯度接近 0→ 模型学不动。
在上面的例子中 $D=2$，$\sqrt{D}=\sqrt{2}\approx1.414$，所以：

$$\frac{QK^T}{\sqrt{2}} \approx \begin{pmatrix} 0.71 & 0 & 0.71 \\ 0.71 & 0.71 & 0 \\ 1.41 & 0.71 & 0.71 \end{pmatrix}$$

第四步：对每一行做 Softmax
Softmax 函数的定义：对一个向量 $\mathbf{z}=(z_1,z_2,\ldots,z_n)$，

$$\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{n} e^{z_j}}$$

它的作用：把一行任意实数变成一个**概率分布**（所有值非负，加起来 =1）。
以第一行 $[0.71, 0, 0.71]$ 为例：

$$e^{0.71} \approx 2.03, \quad e^0 = 1, \quad e^{0.71} \approx 2.03 \tag{6}$$

$$\text{总和} = 2.03 + 1 + 2.03 = 5.06 \tag{7}$$

$$\text{softmax} = \left[\frac{2.03}{5.06}, \frac{1}{5.06}, \frac{2.03}{5.06}\right] = [0.40,\ 0.20,\ 0.40] \tag{8}$$

含义：位置 1 对位置 1 和位置 3 各关注 40%，对位置 2 关注 20%。

第五步：加权求和——乘以 V
设 softmax 之后的注意力矩阵为 A（$N\times N$），最终输出：

$$V' = A\cdot V \quad \text{大小：} (N\times N)\times(N\times M) = N\times M$$

第 i 行的输出：

$$V'_i = \sum_{j=1}^{N} A_{ij}\cdot V_j$$

即用注意力权重对所有位置的 Value 做加权平均。权重大的位置贡献多，权重小的贡献少。

计算复杂度：N^2 的根源

整个过程中，最致命的一步是：

计算 QK^T——这是一个 $N \times N$ 的矩阵！

- **时间：** $\mathcal{O}(N^2 \cdot D)$（N^2 对点积，每个点积 D 次乘法）
- **内存：** $\mathcal{O}(N^2)$（必须存储完整的 $N \times N$ 注意力矩阵）

这就是本文要解决的核心问题！

4.1.2 广义注意力公式

原文翻译

公式(2)实现了一种特定形式的自注意力，称为 softmax 注意力，其中相似度分数是查询和键的点积的指数。给定下标 i 表示矩阵的第 i 行向量，我们可以对任意相似度函数写出如下广义注意力方程：

$$V_i' = \frac{\sum_{j=1}^{N} \mathrm{sim}(Q_i, K_j)\, V_j}{\sum_{j=1}^{N} \mathrm{sim}(Q_i, K_j)} \tag{9}$$

如果我们将相似度函数替换为 $\mathrm{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{D}}\right)$，公式(9)就等价于公式(2)。

公式 (9) 的详细解读

这个公式就是一个**加权平均**——初中就学过！

回忆加权平均： 如果三次考试成绩是 v_1, v_2, v_3，权重是 w_1, w_2, w_3，那加权平均是：

$$\bar{v} = \frac{w_1 v_1 + w_2 v_2 + w_3 v_3}{w_1 + w_2 + w_3}$$

对应到注意力公式：

- $v_j = V_j$ = 第 j 个位置携带的信息（Value）
- $w_j = \mathrm{sim}(Q_i, K_j)$ = 第 i 个 Query 和第 j 个 Key 的相似度（权重）
- 分母 = 所有权重之和（归一化，确保权重加起来为 1）

唯一的约束： $\mathrm{sim}(\cdot,\cdot)$ 必须**非负**（因为权重不能是负数）。

验证：softmax 注意力是特例

令 $\mathrm{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{D}}\right)$（指数函数，永远为正！），代入得：

$$V_i' = \frac{\sum_{j=1}^{N} \exp\left(\frac{Q_i^T K_j}{\sqrt{D}}\right) V_j}{\sum_{j=1}^{N} \exp\left(\frac{Q_i^T K_j}{\sqrt{D}}\right)}$$

分子分母同时出现 exp，这正好就是 softmax 的定义！

4.2 Linearized Attention（线性化注意力）——本文最核心的部分

原文翻译

公式(2)中的注意力定义是通用的，可以用来定义多种其他注意力实现，如多项式注意力或 RBF 核注意力【引用说明：Tsai 等人（2019）在核函数框架下分析了这些不同的注意力形式。再次引用是因为他们提供了将注意力视为核函数的理论基础】。注意，我们需要对 $\mathrm{sim}(\cdot)$ 施加的唯一约束是非负性，这样公式(9)才能定义一个注意力函数。这包括所有核函数 $k(x,y):\mathbb{R}^{2\times F}\to\mathbb{R}_+$。
给定这样一个核函数及其特征表示 $\phi(x)$，我们可以将公式(2)改写如下：

$$V_i' = \frac{\sum_{j=1}^{N}\phi(Q_i)^T\phi(K_j)V_j}{\sum_{j=1}^{N}\phi(Q_i)^T\phi(K_j)} \tag{10}$$

然后利用矩阵乘法的结合律进一步化简为：

$$V_i' = \frac{\phi(Q_i)^T\sum_{j=1}^{N}\phi(K_j)V_j^T}{\phi(Q_i)^T\sum_{j=1}^{N}\phi(K_j)} \tag{11}$$

向量化形式下，上述等式的分子可以更简洁地写为：

$$\left(\phi(Q)\,\phi(K)^T\right)V = \phi(Q)\left(\phi(K)^T V\right)$$

注意特征映射 $\phi(\cdot)$ 是按行（rowwise）作用于矩阵 Q 和 K 的。
从公式(2)可以看出，softmax 注意力的计算成本随 $\mathcal{O}(N^2)$ 增长，其中 N 是序列长度。内存需求同样如此，因为必须存储完整的注意力矩阵以计算对查询、键和值的梯度。相比之下，我们提出的公式(11)中的线性 Transformer 的时间和内存复杂度为 $\mathcal{O}(N)$，因为我们可以先计算 $\sum_{j=1}^{N}\phi(K_j)V_j^T$ 和 $\sum_{j=1}^{N}\phi(K_j)$，然后对每个查询复用它们。

这是全文最重要的公式！

公式(11)是整篇论文的灵魂。理解这个推导，就理解了整篇论文。
下面我们用最慢、最详细的方式，一步一步推导。

核心推导：从 $\mathcal{O}(N^2)$ 到 $\mathcal{O}(N)$

起点：广义注意力公式的分子部分（先不管分母，原理完全一样）

$$\text{分子}_i = \sum_{j=1}^{N}\mathrm{sim}(Q_i,K_j)\cdot V_j$$

第一步：引入核函数分解
假设存在一个函数 $\phi(\cdot)$（叫” 特征映射”），使得：

$$\mathrm{sim}(Q_i,K_j) = \phi(Q_i)^T\phi(K_j)$$

即两个向量的” 相似度” 可以写成：先把它们分别映射到新空间，再做点积。

代入后分子变成:

$$\text{分子}_i = \sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j) \cdot V_j$$

维度说明:

- $\phi(Q_i)$: C 维列向量（C 是映射后的维度）
- $\phi(K_j)$: C 维列向量
- $\phi(Q_i)^T \phi(K_j)$: 1×1 的标量（一个数）
- V_j: M 维行向量
- 所以 $\phi(Q_i)^T \phi(K_j) \cdot V_j$ 是一个 M 维行向量

第二步：关键的”提取公因式”——利用结合律!
注意到 $\phi(Q_i)$ 跟求和变量 j **没有关系**（它只跟 i 有关），所以可以从求和号中”提出来”:

$$\begin{aligned}
\text{分子}_i &= \sum_{j=1}^{N} \underbrace{\phi(Q_i)^T \phi(K_j)}_{\text{标量}} \cdot V_j \\
&= \sum_{j=1}^{N} \phi(Q_i)^T \left(\phi(K_j) V_j^T\right)^T \\
&= \phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j) V_j^T
\end{aligned} \tag{12}$$

让我用更慢的方式解释这一步。
把 $\phi(Q_i)^T \phi(K_j)$ 展开写成分量形式:

$$\phi(Q_i)^T \phi(K_j) = \sum_{c=1}^{C} \phi(Q_i)_c \cdot \phi(K_j)_c$$

那么分子的第 e 个分量（$e = 1, \ldots, M$）是:

$$\begin{aligned}
(\text{分子}_i)_e &= \sum_{j=1}^{N} \left(\sum_{c=1}^{C} \phi(Q_i)_c \cdot \phi(K_j)_c \right) (V_j)_e \\
&= \sum_{c=1}^{C} \phi(Q_i)_c \underbrace{\left(\sum_{j=1}^{N} \phi(K_j)_c \cdot (V_j)_e \right)}_{\text{这个和与}i\text{无关!}}
\end{aligned} \tag{13}$$

关键点：括号内的求和 $\sum_{j=1}^{N} \phi(K_j)_c \cdot (V_j)_e$ 只依赖于 c 和 e，完全不依赖于 i!

所以我们可以**先算好**这个东西（定义为矩阵 G 的 (c,e) 元素）：

$$G_{ce} = \sum_{j=1}^{N} \phi(K_j)_c \cdot (V_j)_e$$

写成矩阵形式：

$$G = \sum_{j=1}^{N} \phi(K_j)\, V_j^T \quad \text{（大小：} C \times M\text{）}$$

然后对任意位置 i：

$$\text{分子}_i = \phi(Q_i)^T\, G$$

第三步：对比两种计算顺序的复杂度
原来的方式（softmax 注意力）：

$$V' = \underbrace{\left(\phi(Q) \cdot \phi(K)^T\right)}_{\substack{(N\times C)\times(C\times N)\\ =N\times N\text{ 矩阵！}}} \cdot V$$

先算 $\phi(Q)\,\phi(K)^T$ 得到 $N \times N$ 矩阵，再乘以 V。
时间复杂度：$\mathcal{O}(N^2C + N^2M) = \mathcal{O}(N^2 \max(C, M))$
新的方式（线性注意力）：

$$V' = \phi(Q) \cdot \underbrace{\left(\phi(K)^T \cdot V\right)}_{\substack{(C\times N)\times(N\times M)\\ =C\times M\text{ 矩阵！}}}$$

先算 $\phi(K)^T V$ 得到 $C \times M$ 矩阵，再乘以 $\phi(Q)$。
时间复杂度：$\mathcal{O}(NCM + NCM) = \mathcal{O}(NCM)$

对比：

	Softmax 注意力	**线性注意力**
中间矩阵大小	$N \times N$	$C \times M$
时间复杂度	$\mathcal{O}(N^2 \max(C, M))$	$\mathcal{O}(NCM)$
内存复杂度	$\mathcal{O}(N^2)$	$\mathcal{O}(N + CM)$

由于 C 和 M 是固定的常数（比如 64），$\mathcal{O}(NCM)$ 本质上就是 $\mathcal{O}(N)$——**线性复杂度**！

数值例子：亲手验证结合律的威力

设 $N = 4, C = 2, M = 2$。

$$\phi(Q) = \begin{pmatrix} 1 & 2 \\ 3 & 1 \\ 2 & 2 \\ 1 & 3 \end{pmatrix}_{4\times 2}, \quad \phi(K) = \begin{pmatrix} 2 & 1 \\ 1 & 3 \\ 3 & 2 \\ 2 & 2 \end{pmatrix}_{4\times 2}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}_{4\times 2}$$

方法 A（原来的方式）：先算 $\phi(Q)\,\phi(K)^T$

$$\phi(Q)\,\phi(K)^T = \begin{pmatrix} 4 & 7 & 7 & 6 \\ 7 & 6 & 11 & 8 \\ 6 & 8 & 10 & 8 \\ 5 & 10 & 9 & 8 \end{pmatrix}_{4\times 4}$$

中间产生了 $\mathbf{4\times 4}$ 的矩阵。

方法 B（新的方式）：先算 $\phi(K)^T V$

$$\phi(K)^T V = \begin{pmatrix} 2 & 1 & 3 & 2 \\ 1 & 3 & 2 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 5 & 6 \\ 3 & 7 \end{pmatrix}_{2\times 2}$$

中间只产生了 $\mathbf{2\times 2}$ 的矩阵！再左乘 $\phi(Q)$ 得到相同的 4×2 结果。
结论：两种方法得到完全一样的结果，但方法 B 的中间矩阵从 4×4 变成了 2×2。当 $N = 10000$ 时，就是从 10000×10000 变成 2×2，节约巨大！

4.2.1 特征映射函数的选择

原文翻译

对于 softmax 注意力，以乘法和加法计量的总成本随 $\mathcal{O}(N^2 \max(D, M))$ 增长，其中 D 是查询和键的维度，M 是值的维度。相反，对于线性注意力，我们首先计算维度为 C 的特征映射。然后，计算新的值需要 $\mathcal{O}(NCM)$ 次加法和乘法。
前面的分析并未考虑核函数和特征函数的选择。注意，对应于指数核的特征函数是无穷维的，这使得精确线性化不可行。然而，可以用随机傅里叶特征来近似它。我们也可以选择具有精确有限维特征映射的核，例如多项式核，它已被证明性能与指数核或 RBF 核相当。2 次多项式核对应的 Transformer 的计算复杂度是 $\mathcal{O}(ND^2M)$。当 $N > D^2$ 时计算复杂度是有利的。注意在实践中这一条件是成立的，因为我们希望能够处理具有数万个元素的序列。
对于我们的实验（处理较短的序列），我们采用如下特征映射，它产生正的相似度函数：

$$\phi(x) = \mathrm{elu}(x) + 1 \tag{14}$$

其中 $\mathrm{elu}(\cdot)$ 表示指数线性单元【引用说明：Clevert 等人（2015）提出了 ELU 激活函数，在负值区域有非零梯度。引用它是因为本文的核心特征映射 ϕ 就基于 ELU】激活函数。我们偏好 $\mathrm{elu}(\cdot)$ 而非 $\mathrm{relu}(\cdot)$，是为了避免在 x 为负时将梯度设置为 0。这一特征映射产生的注意力函数需要 $\mathcal{O}(NDM)$ 次乘法和加法。在我们的实验部分中，我们将展示公式(14)的特征映射与完整 Transformer 性能相当，同时显著降低了计算和内存需求。

公式 (14) 详细解释

ELU 函数的定义：

$$\mathrm{elu}(x) = \begin{cases} x & \text{若} x > 0 \\ e^x - 1 & \text{若} x \leq 0 \end{cases}$$

所以 $\phi(x) = \mathbf{elu}(x) + 1$ 就是：

$$\phi(x) = \begin{cases} x + 1 & \text{若} x > 0 \quad \text{（最小值为 1，当 } x \to 0^+ \text{ 时）} \\ e^x & \text{若} x \leq 0 \quad \text{（值域为 } (0, 1]\text{）} \end{cases}$$

关键性质： $\phi(x) > 0$ 对所有 x 成立！
这很重要，因为注意力权重 $\mathrm{sim}(Q_i, K_j) = \phi(Q_i)^T \phi(K_j)$ 需要非负。如果 ϕ 的每个分量都大于 0，那它们的点积一定大于 0。
为什么不用 ReLU?

$$\mathrm{ReLU}(x) = \max(0, x) \implies \phi(x) = \max(0, x) + 1 = \begin{cases} x + 1 & x > 0 \\ 1 & x \leq 0 \end{cases}$$

问题：当 $x \leq 0$ 时 ReLU 的梯度为 0。ELU 在 $x \leq 0$ 时梯度是 $e^x > 0$，可以继续学习。

4.3 Causal Masking（因果掩码）

原文翻译

Transformer 架构可以通过对注意力计算进行掩码来高效训练自回归模型，使得第 i 个位置只能受到位置 j（当且仅当 $j \leq i$）的影响，即一个位置不能受到后续位置的影响。形式化地，这种因果掩码将公式(9)修改如下：

$$V_i' = \frac{\sum_{j=1}^{i} \mathrm{sim}(Q_i, K_j)\, V_j}{\sum_{j=1}^{i} \mathrm{sim}(Q_i, K_j)} \tag{15}$$

按照 § 3.2 的推理，我们将掩码注意力线性化如下：

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)\, V_j^T}{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)} \tag{16}$$

通过引入 S_i 和 Z_i：

$$S_i = \sum_{j=1}^{i} \phi(K_j)\, V_j^T \tag{17}$$

$$Z_i = \sum_{j=1}^{i} \phi(K_j) \tag{18}$$

我们可以将公式(16)简化为:

$$V_i' = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i} \tag{19}$$

注意，S_i 和 Z_i 可以从 S_{i-1} 和 Z_{i-1} 在常数时间内计算得到，因此使得带因果掩码的线性 Transformer 的计算复杂度相对于序列长度为线性。

因果掩码的线性化——超详细推导

什么是因果掩码?
在自回归生成中（比如逐像素画图），生成第 3 个像素时，只能参考第 1、2、3 个像素，**不能偷看**第 4、5、... 个像素。
数学上：求和的上限从 N 变成了 i（只看 $j=1$ 到 $j=i$）。

问题：在第4.2节的线性注意力中，我们把 $\sum_{j=1}^{N} \phi(K_j) V_j^T$ 提前算好，然后复用。但现在每个位置 i 的求和上限不同!
解决方案：用累加（cumulative sum）!
定义两个状态量:

$$S_i = \sum_{j=1}^{i} \phi(K_j) V_j^T \quad (C \times M \text{ 矩阵——”注意力记忆”}) \tag{20}$$

$$Z_i = \sum_{j=1}^{i} \phi(K_j) \quad (C \text{ 维向量——”归一化记忆”}) \tag{21}$$

核心性质——递推关系：

$$S_i = S_{i-1} + \phi(K_i) V_i^T \tag{22}$$
$$Z_i = Z_{i-1} + \phi(K_i) \tag{23}$$

逐步验证：

$$S_1 = \phi(K_1) V_1^T \tag{24}$$
$$S_2 = \phi(K_1) V_1^T + \phi(K_2) V_2^T = S_1 + \phi(K_2) V_2^T \quad \checkmark \tag{25}$$
$$S_3 = S_2 + \phi(K_3) V_3^T \quad \checkmark \tag{26}$$
$$\vdots \tag{27}$$

计算代价：从 S_{i-1} 到 S_i，只需加上一个 $C \times M$ 的矩阵，代价 $\mathcal{O}(CM)$——**与 i 无关，是常数!**

然后第 i 个位置的输出:

$$V_i' = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}$$

这是一个 C 维向量与 $C \times M$ 矩阵的乘法，代价也是 $\mathcal{O}(CM)$。
总复杂度：N 个位置，每个位置 $\mathcal{O}(CM) \Rightarrow$ 总共 $\mathcal{O}(NCM) = \mathcal{O}(N)$。

4.3.1 梯度计算

原文翻译

对公式(19)的朴素实现（在任何深度学习框架中）需要存储所有中间值 S_i 以计算梯度。这会使内存消耗增加 $\max(D, M)$ 倍，因此阻碍了因果线性注意力在更长序列或更深模型上的应用。为此，我们将公式(16)中分子的梯度推导为累加和的形式。这使我们能够在**线性时间**和**常数内存**下计算因果线性注意力的前向和反向传播。详细推导在补充材料中给出。

给定分子 $\bar{V}_i$ 以及标量损失函数关于分子的梯度 $\nabla_{\bar{V}_i}\mathcal{L}$，我们推导出 $\nabla_{\phi(Q_i)}\mathcal{L}$、$\nabla_{\phi(K_i)}\mathcal{L}$ 和 $\nabla_{V_i}\mathcal{L}$ 如下：

$$\nabla_{\phi(Q_i)}\mathcal{L} = \nabla_{\bar{V}_i}\mathcal{L}\left(\sum_{j=1}^{i}\phi(K_j)\,V_j^T\right)^T \tag{28}$$

$$\nabla_{\phi(K_i)}\mathcal{L} = \left(\sum_{j=i}^{N}\phi(Q_j)\left(\nabla_{\bar{V}_j}\mathcal{L}\right)^T\right)V_i \tag{29}$$

$$\nabla_{V_i}\mathcal{L} = \left(\sum_{j=i}^{N}\phi(Q_j)\left(\nabla_{\bar{V}_j}\mathcal{L}\right)^T\right)^T\phi(K_i) \tag{30}$$

公式(16)和(28)–(30)中的累加和项在线性时间内计算，并且相对于序列长度只需要常数内存。这使得算法的计算复杂度为 $\mathcal{O}(NCM)$，内存为 $\mathcal{O}(N\max(C, M))$（给定维度为 C 的特征映射）。前向和反向传播分子部分的伪代码在算法 1中给出。

梯度推导——超详细版

设定：为简化符号，假设 Q 和 K 已经过 ϕ 映射。分子为：

$$\bar{V}_{ie} = \sum_{d=1}^{D} Q_{id}\sum_{j=1}^{i} K_{jd}V_{je} = \sum_{d=1}^{D}\sum_{j=1}^{i} Q_{id}K_{jd}V_{je}$$

符号说明：

- i：位置索引，$1 \le i \le N$
- e：Value 的分量索引，$1 \le e \le M$
- d：Query/Key 的分量索引，$1 \le d \le D$
- $\bar{V}_{ie}$：第 i 个位置输出（分子部分）的第 e 个分量

（一）关于 Q 的梯度

目标：求 $\dfrac{\partial\mathcal{L}}{\partial Q_{lt}}$（$\mathcal{L}$ 对第 l 个 Query 的第 t 个分量的偏导）

用链式法则：

$$\frac{\partial\mathcal{L}}{\partial Q_{lt}} = \sum_{\substack{\text{所有被}\\ Q_{lt}\text{影响的输出}}} \frac{\partial\mathcal{L}}{\partial\bar{V}_{ie}}\cdot\frac{\partial\bar{V}_{ie}}{\partial Q_{lt}}$$

问题：Q_{lt} 影响了哪些 $\bar{V}_{ie}$？
看公式 $\bar{V}_{ie} = \sum_{d=1}^{D} \sum_{j=1}^{i} Q_{id} K_{jd} V_{je}$，其中出现了 Q_{id}。当且仅当 $i = l$ 且 $d = t$ 时，Q_{lt} 才出现在公式中。
所以 Q_{lt} **只影响** $\bar{V}_{l,1}, \bar{V}_{l,2}, \ldots, \bar{V}_{l,M}$（只影响同一行！）
因此：

$$\frac{\partial \mathcal{L}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \cdot \frac{\partial \bar{V}_{le}}{\partial Q_{lt}}$$

接下来计算 $\frac{\partial \bar{V}_{le}}{\partial Q_{lt}}$：

$$\bar{V}_{le} = \sum_{d=1}^{D} \sum_{j=1}^{l} Q_{ld} K_{jd} V_{je} \tag{31}$$

$$\frac{\partial \bar{V}_{le}}{\partial Q_{lt}} = \sum_{j=1}^{l} K_{jt} V_{je} \quad \text{（只有 } d = t \text{ 的项有贡献）} \tag{32}$$

代入：

$$\boxed{\frac{\partial \mathcal{L}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \left(\sum_{j=1}^{l} K_{jt} V_{je} \right)}$$

写成矩阵形式：

$$\boxed{\nabla_{Q_i} \mathcal{L} = \nabla_{\bar{V}_i} \mathcal{L} \cdot \left(\sum_{j=1}^{i} K_j V_j^T \right)^T = \nabla_{\bar{V}_i} \mathcal{L} \cdot S_i^T}$$

S_i 是正向累加和——和前向传播一样从前往后累加即可！

（二）关于 K 的梯度

K_{lt} 出现在 $\bar{V}_{ie}$ 中的条件：$j = l$ 且 $d = t$，且 $i \geq l$（因为求和 j 从 1 到 i，只有 $i \geq l$ 时才包含 $j = l$）。
所以 K_{lt} 影响 $\bar{V}_{l,\cdot}, \bar{V}_{l+1,\cdot}, \ldots, \bar{V}_{N,\cdot}$（影响从 l 到 N 的所有行！）

$$\frac{\partial \mathcal{L}}{\partial K_{lt}} = \sum_{e=1}^{M} \sum_{i=l}^{N} \frac{\partial \mathcal{L}}{\partial \bar{V}_{ie}} \cdot \frac{\partial \bar{V}_{ie}}{\partial K_{lt}}$$

计算 $\frac{\partial \bar{V}_{ie}}{\partial K_{lt}}$（当 $i \geq l$ 时）：

$$\frac{\partial \bar{V}_{ie}}{\partial K_{lt}} = Q_{it} V_{le}$$

代入并写成矩阵形式：

$$\boxed{\nabla_{K_i} \mathcal{L} = \left(\sum_{j=i}^{N} Q_j (\nabla_{\bar{V}_j} \mathcal{L})^T \right) V_i}$$

这是一个**反向累加和**——从 N 到 i 累加。类似 RNN 中的” 反向传播通过时间”(BPTT)！

（三）关于 V 的梯度

同理，V_{le} 影响所有 $i \geq l$ 的输出。推导过程类似：

$$\boxed{\nabla_{V_i}\mathcal{L} = \left(\sum_{j=i}^{N} Q_j (\nabla_{\bar{V}_j}\mathcal{L})^T\right)^T K_i}$$

总结：

- ∇_Q：正向累加和 $\rightarrow$ 从前往后扫一遍，$\mathcal{O}(N)$
- ∇_K, ∇_V：反向累加和 $\rightarrow$ 从后往前扫一遍，$\mathcal{O}(N)$
- 每步累加代价：$\mathcal{O}(CM)$（常数）
- 总梯度计算：$\mathcal{O}(NCM)$，内存 $\mathcal{O}(N\max(C, M))$

4.3.2 训练与推理

原文翻译

在训练自回归 Transformer 模型时，完整的真实序列（ground truth）是可用的。这使得分层并行计算成为可能——既适用于公式(1)中的 $f_l(\cdot)$，也适用于注意力计算。因此，Transformer 的训练效率高于循环神经网络。另一方面，在推理过程中，时间步 i 的输出是时间步 $i+1$ 的输入。这使得自回归模型不可能并行化。此外，Transformer 每个时间步的成本不是固定的；相反，它随当前序列长度的平方增长，因为必须为所有先前时间步计算注意力。
我们提出的线性 Transformer 模型**兼具两者之长**。在训练方面，计算可以并行化并充分利用 GPU 或其他加速器。在推理方面，我们模型的每次预测的时间和内存成本是常数。这意味着我们可以简单地将 $\phi(K_j)\,V_j^T$ 矩阵存储为内部状态，并在每个时间步像循环神经网络一样更新它。这使得推理比其他 Transformer 模型**快数千倍**。

4.4 Transformers are RNNs（Transformer 就是 RNN!）

原文翻译

在文献中，Transformer 模型被认为是与循环神经网络根本不同的方法。然而，从 § 3.3 中的因果掩码表述以及前一节的讨论可以明显看出，任何带因果掩码的 Transformer 层都可以写成一个模型——给定输入，修改内部状态然后预测输出——即循环神经网络（RNN）。注意，与 Universal Transformer【引用说明：Dehghani 等人（2018）提出的 Universal Transformer 在深度维度上引入了递归。这里引用是为了做区分——本文讨论的递归是关于时间的递归，而非深度递归】不同，我们考虑的递归是关于时间的，而不是关于深度的。
在下面的等式中，我们将公式(1)的 Transformer 层形式化为循环神经网络。所得

的 RNN 有两个隐藏状态，分别是注意力记忆 s 和归一化记忆 z。我们用下标表示递归中的时间步。

$$s_0 = 0 \tag{33}$$
$$z_0 = 0 \tag{34}$$
$$s_i = s_{i-1} + \phi(x_i W_K)\,(x_i W_V)^T \tag{35}$$
$$z_i = z_{i-1} + \phi(x_i W_K) \tag{36}$$
$$y_i = f_l\left(\frac{\phi(x_i W_Q)^T s_i}{\phi(x_i W_Q)^T z_i} + x_i\right) \tag{37}$$

在上面的等式中，x_i 表示第 i 个输入，y_i 表示特定 Transformer 层的第 i 个输出。注意我们的表述对特征函数不施加任何约束，理论上它可以用来表示**任何** Transformer 模型，甚至使用 softmax 注意力的模型。这一表述是朝着更好地理解 Transformer 与流行的循环网络【引用说明：Hochreiter & Schmidhuber（1997）提出了 LSTM，长短期记忆网络。引用它是因为 LSTM 是最著名的 RNN 变体，本文的 RNN 表述提供了一种理解 Transformer 与 LSTM 之间关系的新视角】以及信息存储和检索所用过程之间关系的第一步。

公式 (33)–(37)：RNN 形式逐行解读

这五行公式完整地定义了一个 RNN。让我逐行翻译成大白话：

公式 (33)–(34)：初始化

$$s_0 = \mathbf{0}_{C\times M}, \qquad z_0 = \mathbf{0}_C$$

” 在一切开始之前，记忆是空的。”

公式 (35)：更新注意力记忆

$$s_i = s_{i-1} + \underbrace{\phi(x_i W_K)}_{\text{新 Key 的特征映射}}\ \underbrace{(x_i W_V)^T}_{\text{新 Value 的转置}}$$

翻译：把新来的” 键-值对” 记录到记忆中。

- $x_i W_K$：把第 i 个输入投影为 Key
- $\phi(x_i W_K)$：对 Key 做特征映射（得到 C 维向量）
- $x_i W_V$：把第 i 个输入投影为 Value（得到 M 维向量）
- $\phi(x_i W_K)\,(x_i W_V)^T$：外积，得到一个 $C \times M$ 矩阵
- $s_i = s_{i-1} + \cdots$：加到之前的记忆上

直觉：就像在笔记本上追加了一条新笔记。笔记本的大小始终是 $C \times M$，固定不变。

公式 (36)：更新归一化记忆

$$z_i = z_{i-1} + \phi(x_i W_K)$$

翻译：记录到目前为止累积了多少个 Key 特征。用于后面的归一化。

公式 (37)：根据记忆产生输出

$$y_i = f_l\left(\underbrace{\frac{\phi(x_iW_Q)^T s_i}{\phi(x_iW_Q)^T z_i}}_{\text{注意力输出}} + \underbrace{x_i}_{\text{残差连接}}\right)$$

翻译：

1. 把第 i 个输入投影为 Query：x_iW_Q
2. 对 Query 做特征映射：$\phi(x_iW_Q)$（C 维向量）
3. 用 Query 去查询记忆：$\phi(x_iW_Q)^T s_i$（C 维 $\times$ $C\times M$ 矩阵 $=$ M 维向量）
4. 归一化：除以 $\phi(x_iW_Q)^T z_i$
5. 加上残差连接：$+x_i$
6. 过前馈网络：$f_l(\cdot)$

为什么这意味着”Transformer 就是 RNN”？

经典 RNN 的定义：

$$h_i = g(h_{i-1}, x_i), \qquad y_i = o(h_i, x_i)$$

即：给定上一步的隐藏状态 h_{i-1} 和当前输入 x_i，更新隐藏状态，然后产生输出。
线性 Transformer 完全符合这个定义：

- **隐藏状态：** (s_i, z_i)——一个 $C \times M$ 矩阵和一个 C 维向量
- **状态更新规则：** $s_i = s_{i-1} + \phi(x_iW_K)\,(x_iW_V)^T$，$z_i = z_{i-1} + \phi(x_iW_K)$
- **输出规则：** $y_i = f_l(\phi(x_iW_Q)^T s_i / \phi(x_iW_Q)^T z_i + x_i)$
- **每步计算量：** $\mathcal{O}(CM)$——**与时间步 i 无关！**

”两全其美”的模型：

	标准 Transformer	标准 RNN	线性 Transformer
训练	可并行（快）✓	不可并行（慢）×	可并行（快）✓
推理	$\mathcal{O}(N^2)$（慢）×	$\mathcal{O}(1)$ 每步 ✓	$\mathcal{O}(1)$ 每步 ✓

训练时：用矩阵形式 $V' = \phi(Q)\,(\phi(K)^T V)$ 并行计算所有位置（像 Transformer）。
推理时：用递推形式 $s_i = s_{i-1} + \cdots$ 逐步计算（像 RNN），每步常数时间！

5 Experiments（实验）

原文翻译

在本节中，我们实验分析所提线性 Transformer 的性能。首先，在 § 4.1 中，我们在合成数据上评估线性化注意力的计算成本、内存消耗和收敛性。为了进一步展示线性 Transformer 的有效性，我们在两个实际应用上评估我们的模型：§ 4.2 的图像生成和 § 4.3 的自动语音识别。我们表明，我们的模型在性能上与最先进的 Transformer 架构具有竞争力，同时需要的 GPU 内存和计算量显著更少。
在所有实验中，我们将模型与两个基线进行比较：使用 softmax 注意力的完整 Transformer 和 Reformer【引用说明：Kitaev 等人（2020），本文的主要对比基线】——后者是最先进的加速 Transformer 架构。对于 Reformer，我们使用已发布代码的 PyTorch 重新实现；对于完整 Transformer，我们使用 PyTorch【引用说明：Paszke 等人（2019），引用 PyTorch 框架是因为所有实验都基于此实现】的默认实现。注意，对于 Reformer 我们没有使用可逆层（reversible layers），但这不影响结果，因为我们只测量与自注意力层相关的内存消耗。在所有实验中，我们使用 **softmax** 指代标准 Transformer 架构，**linear** 指代我们提出的线性 Transformer，**lsh-X** 指代 Reformer，其中 X 表示哈希轮数。
对于线性 Transformer 的训练，我们使用公式(14)中的特征映射。我们的 PyTorch 代码、文档和示例可以在https://linear-transformers.com/找到。公式(28)–(30)中的常数内存梯度计算用大约 200 行 CUDA 代码实现。

5.1 合成任务

5.1.1 收敛性分析

原文翻译

为了检验线性 Transformer 的收敛性质，我们在一个带因果掩码的人工复制任务上训练。即 Transformer 必须复制一系列符号，类似于 Kitaev 等人【引用说明：Reformer 论文中也使用了类似的序列复制任务来验证其方法】的序列复制任务。我们使用最大长度 128 的序列，包含 10 种不同的符号，由一个专用的分隔符号分隔。对于所有三种方法，我们训练一个 4 层、8 个注意力头的 Transformer，批大小为 64，使用 RAdam 优化器【引用说明：Liu 等人（2019）提出了 RAdam，一种带方差修正的 Adam 变体，能在训练初期提供更稳定的优化。引用它是因为所有实验都使用了这个优化器】，学习率为 10^{-3}，在 3000 次更新后降低到 10^{-4}。图 1展示了损失与梯度步数的关系。我们观察到 linear 平滑收敛，达到了比 lsh 更低的损失值（因为没有哈希引入的噪声）。特别地，它达到了与 softmax 相同的损失。

图 1 解读

任务：给模型一个符号序列（10 种符号，最长 128 个），让它逐个复制输出。这测试的是模型” 记住之前看到的内容” 的能力。
观察：

- **Linear** 和 **Softmax** 收敛到几乎相同的损失值
- **LSH/Reformer** 收敛到更高的损失（表现更差）

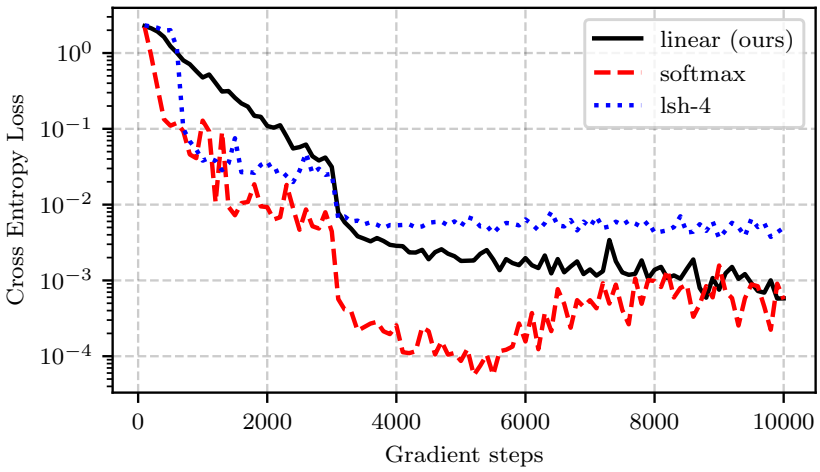


Figure 1: 序列复制任务上的收敛曲线对比（原论文 Figure 2）

- Linear 的曲线非常平滑，而 LSH 的曲线有噪声（因为哈希本身引入随机性）

结论：线性注意力的表达能力足够完成这个任务，性能和标准 softmax 注意力一样好。

5.1.2 计算和内存需求

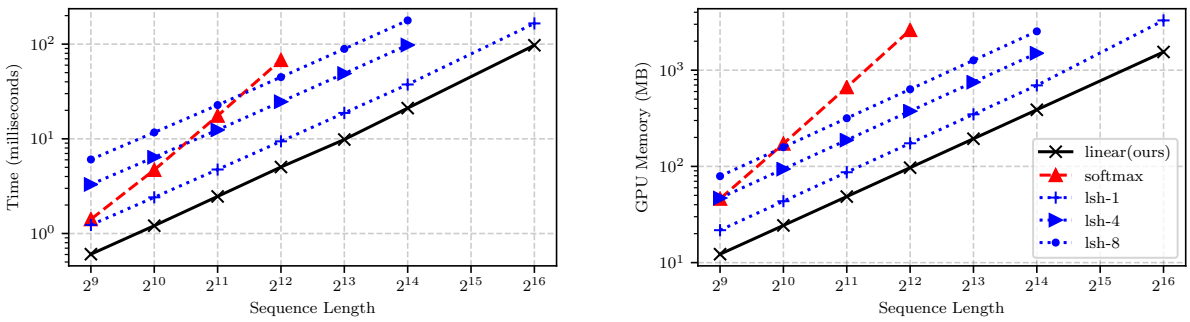
原文翻译

在本小节中，我们比较 Transformer 在计算和内存需求方面的表现。我们对不同序列长度 $N \in \{2^9, 2^{10}, \ldots, 2^{16}\}$ 的合成输入计算注意力及其梯度，并测量每种 Transformer 变体的峰值 GPU 内存分配和所需时间。我们将批大小与序列长度成反比地缩放，并报告批内每个样本的时间和内存。
每种方法评估到 GPU 内存能容纳的最大序列长度。在此基准测试中我们使用一块拥有 11GB 内存的 NVidia GTX 1080 Ti。结果是：softmax 的最大序列长度为 4,096 个元素，lsh-4 和 lsh-8 为 16,384。正如预期的，softmax 相对于序列长度呈二次方增长。如图 2所示，我们的方法在每种配置下都比基线更快且需要更少的内存。我们观察到 Reformer 和线性注意力都相对于序列长度呈线性增长。注意，虽然 Reformer 的渐近复杂度为 $\mathcal{O}(N \log N)$，但 $\log N$ 足够小，不会影响计算时间。

图 2 解读

横轴是序列长度 N（从 $2^9 = 512$ 到 $2^{16} = 65536$），纵轴是计算时间/内存。
关键观察：

- **Softmax（红色）**：明显的向上弯曲——二次方增长 $\mathcal{O}(N^2)$。在 $N = 4096$ 时就超出 GPU 内存了！
- **Linear（蓝色）**：近乎直线——线性增长 $\mathcal{O}(N)$。即使 $N = 65536$ 也没问题。
- **Reformer**：也接近线性（理论 $\mathcal{O}(N \log N)$，但 $\log N$ 很小），不过在每个 N 值上都比 Linear 更慢、更耗内存。



(a) 计算时间对比 (b) 内存消耗对比

Figure 2: 不同序列长度下的计算时间和内存消耗对比（原论文 Figure 1）

这张图直观地展示了 $\mathcal{O}(N^2)$ 和 $\mathcal{O}(N)$ 的巨大差距。

5.2 图像生成

原文翻译——图像生成引言

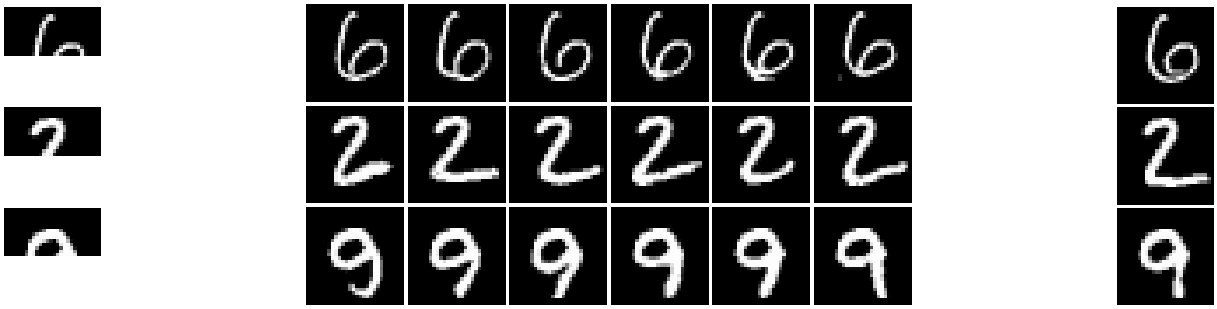
Transformer 在条件或无条件自回归生成任务【引用说明：Radford 等人（2019）即 GPT-2 展示了 Transformer 强大的自回归生成能力；Child 等人（2019）即 Sparse Transformer 将其应用于长序列生成。引用这两篇说明 Transformer 已在生成任务中表现出色】上取得了很好的结果，但由于任务本身固有的顺序性和内存随序列长度平方增长的特点，从 Transformer 中采样是很慢的。在本节中，我们训练带因果掩码的 Transformer 逐像素预测图像。我们在比特/维度（bits per dimension）方面达到的性能与 softmax 注意力相当，同时能够以**超过 1000 倍**的速度生成图像，并且**每张图像的内存恒定**——从第一个像素到最后一个像素。读者可参考我们的补充材料以了解训练演化对比、生成图像的质量比较以及生成单张图像的时间。此外，我们还与一种更快的 softmax Transformer 进行了比较，该 Transformer 在推理时缓存了键和值。

5.2.1 MNIST

原文翻译

首先，我们在广泛使用的 MNIST 数据集【引用说明：LeCun 等人（2010），手写数字识别的经典基准数据集】上评估自回归 Transformer 的图像生成。本实验的架构包含 8 个注意力层，每层 8 个注意力头。我们将嵌入维度设为 256，即每个头 32 维。前馈层的维度是嵌入维度的 4 倍。我们使用由 Salimans 等人【引用说明：即 PixelCNN++（2017），引入了混合 logistics 输出分布来建模像素值。本文使用了这一建模方式】引入的 10 个 logistics 混合分布来建模输出。我们使用 RAdam 优化器，学习率为 10^{-4}，所有模型训练 250 个 epoch。对于 Reformer 基线，我们使用 1 轮和 4 轮哈希。此外，如 Kitaev 等人所建议的，我们使用 64 个桶（buckets），每个块（chunk）大约包含 32 个元素。具体来说，我们将 783 个元素长的输入序列分成 27 个块，每块 29 个元素。由于序列长度相对较小（仅 784 个像素），为消除不

无条件生成样本



(a) 遮挡 (b) 补全结果 (c) 原图

Figure 3: Linear Transformer 在 MNIST 上的生成结果（原论文 Figure 3）。(a) 遮挡原图下半部分，(b) 模型补全结果，(c) 原始图片。

同批大小带来的差异，我们对所有方法统一使用批大小 10。
表 ??总结了结果。我们观察到，线性 Transformer 在最终困惑度方面几乎达到了与 softmax Transformer 相同的性能，同时生成图像的速度快了 300 多倍。这得益于我们模型的低内存需求——它能够用单块 GPU 同时生成 10,000 张 MNIST 图像。特别地，内存相对于序列长度是恒定的，因为像素之间唯一需要存储的只有公式(35)和(36)中所描述的 s_i 和 z_i 值。另一方面，softmax 和 Reformer 都需要随序列长度增长的内存。
我们的 MNIST 模型的图像补全和无条件样本如图 3所示。我们观察到，线性 Transformer 生成了非常令人信服的样本，边界清晰且没有噪声。在图像补全方面，我们还观察到 Transformer 学会了使用与原图相同的笔触风格和宽度，有效地关注了长时间距离的信息。注意，由于所有模型达到的困惑度大致相同，我们在不同模型生成的样本之间没有观察到定性差异。

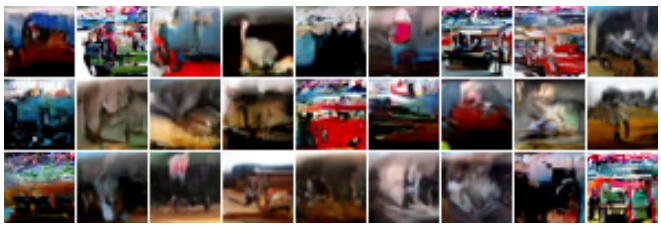
MNIST 实验解读

任务： 用自回归 Transformer 逐像素生成 28×28 的手写数字图片。序列长度 $N = 784$。
结果：

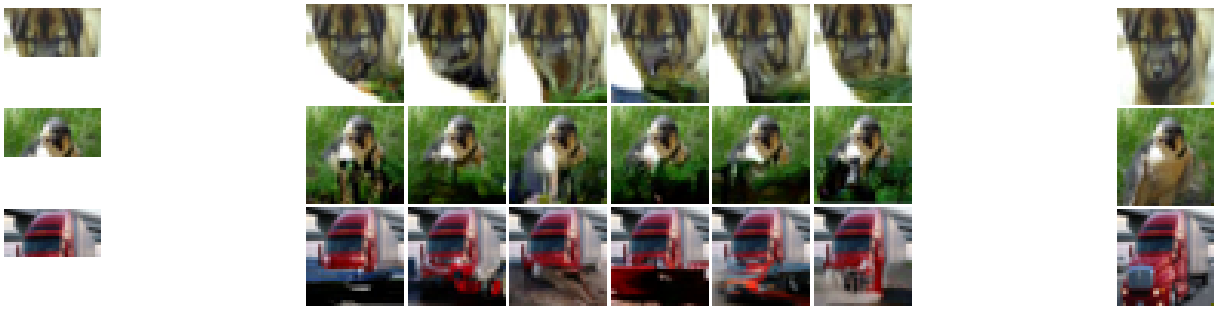
方法	Bits/dim↓	生成速度（图/秒）↑	
Softmax	0.621	0.45	(1×)
LSH-1	0.745	0.68	(1.5×)
LSH-4	0.676	0.27	(0.6×)
Linear（本文）	**0.644**	**142.8**	**(317×)**

Bits/dim 是衡量生成模型质量的指标，越低越好。
Linear 的 0.644 vs Softmax 的 0.621：质量非常接近。
Linear 的 142.8 图/秒 vs Softmax 的 0.45 图/秒：速度快了 **317 倍**！
为什么能这么快？ 因为推理时使用 RNN 形式，每生成一个像素只需更新固定大小的状态 (s_i, z_i)，可以同时生成 10000 张图片！

5.2.2 CIFAR-10



无条件生成样本



(a) 遮挡 (b) 补全结果 (c) 原图

Figure 4: Linear Transformer 在 CIFAR-10 上的生成结果（原论文 Figure 4）

原文翻译

随着序列长度的增加，我们线性化表述的优势也在增大。为了展示这一点，我们训练 16 层 Transformer 来生成 CIFAR-10 图像【引用说明：Krizhevsky 等人（2009），CIFAR-10 是一个包含 10 类自然图像的经典基准数据集】。每层使用与前一个实验相同的配置。对于 Reformer，我们再次使用 64 个桶和 83 个块（每块 37 个元素，大约是论文建议的 32 个）。由于序列长度几乎是前一个实验的 4 倍，完整 Transformer 在我们可用的最大 GPU（24GB 内存的 NVidia P40）上只能使用批大小 1。线性 Transformer 和 Reformer 均使用批大小 4。所有模型训练 7 天。我们在表 **??**中以比特/维度和图像生成吞吐量报告结果。注意，虽然本实验的重点不在最终困惑度，但显然随着序列长度的增长，快速 Transformer 模型在每 GPU 小时上变得越来越高效，取得了优于慢速模型的分数。
由于对于 Reformer 和 softmax 注意力，生成单个像素的内存和时间随像素数的平方增长，我们线性 Transformer 吞吐量的提升更加显著。特别地，**对于 softmax Transformer 生成的每一张图像，我们的方法可以生成 4,460 张图像**。我们模型的图像补全和无条件样本如图 4所示。我们观察到，模型生成的图像具有空间一致性，并且能够令人信服地补全图像，而不会显著妨碍对图像类别的识别。例如，在

图 4(b) 中，所有图像都成功补全了狗的鼻子（第一行）或卡车的挡风玻璃（最后一行）。

CIFAR-10 实验解读

CIFAR-10 的图片是 $32 \times 32 \times 3$（彩色），序列长度 $N = 3072$，是 MNIST 的约 4 倍。

结果：

方法	Bits/dim↓	生成速度（图/秒）↑	
Softmax	3.47	0.004	(1×)
LSH-1	3.39	0.015	(3.75×)
LSH-4	3.51	0.005	(1.25×)
Linear（本文）	**3.40**	**17.85**	(**4462**×)

关键发现：

- 加速比从 MNIST 的 317× 增大到 CIFAR-10 的 **4462**×——序列越长，优势越大！
- Linear 的 bits/dim (3.40) 甚至比 Softmax (3.47) 更好！原因：在固定 7 天训练时间内，Linear 完成了更多 epoch。
- Softmax 在最大的 GPU（24GB）上也只能用 batch size=1，而 Linear 可以用 4。

图像补全质量（图4）：模型看到图片上半部分后，能合理地补全下半部分（比如补全狗的鼻子、卡车的挡风玻璃等）。

5.3 语音识别

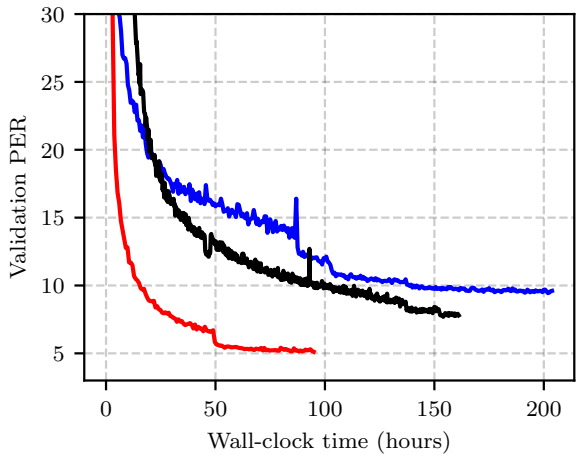


Figure 5: 语音识别任务的训练曲线（原论文补充材料）

原文翻译

为了证明我们的方法也可以用于非自回归任务，我们使用连接时序分类（Connectionist Temporal Classification, CTC）损失【引用说明：Graves 等人（2006）提出了 CTC 损失函数，使得端到端序列标注成为可能，无需预先对齐输入输出。引用它是因为语音识别实验使用了这一损失函数】评估线性 Transformer 在端到端自动语音识别中的性能。在此设置下，我们以非自回归方式为每个输入帧预测音素的分布。我们使用 80 小时的 WSJ 数据集【引用说明：Paul & Baker（1992），Wall Street Journal 语料库是语音识别的经典基准数据集】，以 40 维梅尔标度滤波器组（mel-scale filterbanks）作为特征，不使用时间差分。数据集中序列的平均长度为 800 帧，最大序列长度为 2,400 帧。

对于此任务，我们还与一个双向 LSTM【引用说明：Hochreiter & Schmidhuber（1997），此处引用 LSTM 是因为它是语音识别中常用的 RNN 基线】进行比较，该 LSTM 有 3 层，隐藏维度为 320。我们使用 Adam 优化器【引用说明：Kingma & Ba（2014）提出了 Adam 优化器。引用它是因为 LSTM 基线使用了此优化器】，学习率为 10^{-3}，当验证误差停止下降时降低学习率。对于 Transformer 模型，我们使用 9 层、6 个头，嵌入维度与图像实验相同。优化器使用 RAdam，初始学习率为 10^{-4}，当验证误差停止下降时除以 2。

所有模型在音素错误率（PER）和每 epoch 训练时间方面进行评估。我们观察到 linear 在性能和速度两方面都以大幅优势超过了循环网络基线和 Reformer，如表 ??所示。注意，softmax Transformer 与所有基线相比达到了更低的音素错误率，但速度显著更慢。特别地，线性 Transformer 每 epoch 快了 **3 倍以上**。我们在补充材料中提供了训练演化图。

语音识别实验解读

任务：端到端语音识别——输入音频帧，输出音素。使用 WSJ 数据集，平均序列长度 800 帧，最长 2400 帧。这是一个**非自回归**任务（使用 CTC 损失）。

结果：

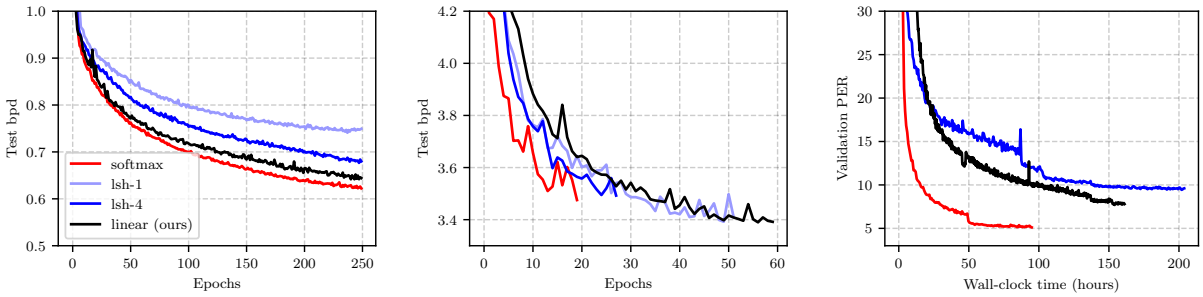
方法	验证集 PER↓	每 epoch 时间（秒）↓
Bi-LSTM	10.94	1047
Softmax	5.12	2711
LSH-4	9.33	2250
Linear（本文）	8.08	**824**

PER = 音素错误率（Phoneme Error Rate），越低越好。

结论：

- Linear 大幅优于 LSTM（8.08 vs 10.94）和 Reformer（8.08 vs 9.33）
- Linear 训练速度最快——比 Softmax 快 3 倍以上（824 秒 vs 2711 秒）
- 但 Softmax 的精度仍然最高（5.12）——这说明在某些任务上，线性注意力的近似确实会损失一些表达能力

5.4 训练演化对比（补充材料）



(a) MNIST (b) CIFAR-10 (c) 语音识别

Figure 6: 所有实验的训练演化曲线（原论文补充材料 Figure 5）

原文翻译

在图 6中, 我们展示了所有实验中所有 Transformer 模型的训练演化。对于 MNIST 实验（图 6a），我们对所有方法训练 250 个 epoch。序列长度足够小，使得所有方法的训练时间不会有显著差异。我们观察到我们的方法在收敛上与 softmax 注意力相当，显著优于两种 Reformer 变体。
另一方面，对于 CIFAR-10（图 6b），我们对所有方法训练固定时间，即 7 天。我们观察到 lsh-1 和 linear 完成了显著更多的 epoch（多于 softmax 和 lsh-4），并取得了更好的性能。这一差距预计会随着序列长度的进一步增加而扩大。
最后，在语音识别实验（图 6c）中，softmax 在收敛性方面显著优于 Reformer 和 linear。注意 linear 每 epoch 快 3 倍，这意味着它完成的 epoch 数约为 softmax 的 4 倍。尽管 softmax 注意力在此任务上更好，但我们观察到线性 Transformer 在收敛性和最终性能方面都显著优于 Reformer。

图 6 解读

MNIST（左）：按 epoch 数训练 250 轮。Linear 和 Softmax 曲线几乎完全重合，都优于 Reformer。
CIFAR-10（中）：按**时间**训练 7 天。Linear 和 LSH-1 完成了更多 epoch（因为每轮更快），因此达到了更好的性能。
语音识别（右）：所有方法训练至收敛。Softmax 最终性能最好，但 Linear 训练速度快 3 倍。

5.5 补充材料：图像生成吞吐量讨论

原文翻译——有状态 softmax 基线

在主论文 § 4.2 中，我们报告了图像生成吞吐量并与 softmax Transformer 和 lsh 进行了比较。在本节中，我们创建了另一个基线，记为 **stateful-softmax**，它将 softmax 自回归 Transformer 实现为循环模型。即所有键和值都被保存，然后在预测序列的下一个元素时再次传递给模型。这种循环模型的状态是键和值的集合，其大小与序列长度成正比。这与我们提出的模型有本质区别——我们的模型具有固定维度的状态，并且给定前一个状态计算第 i 个状态的计算成本固定，与 i 无关。

我们观察到 stateful-softmax 显著快于标准 Transformer。然而，它的复杂度仍然是关于序列长度的二次方，我们的表述在 CIFAR-10 上快了超过 50 倍。此外，我们想指出，为 Reformer 实现类似的有状态注意力并非简单的任务，因为排序和分块操作需要在每次新输入时重新执行。

原文翻译——等化批大小

在前面的章节中，我们评估了自回归图像生成任务中所有 Transformer 变体的吞吐量。然而，另一个需要考虑的重要因素是延迟（latency），即生成单张图像所需的总时间。为此，我们使用批大小 1 并测量所有方法生成单张图像所需的时间。除了在 GPU 上运行推理外，我们还评估了在 CPU 上所需的时间。
我们观察到，所有方法都无法充分利用 GPU，生成单张图像的吞吐量显著低于批量生成。所提出的线性 Transformer 比所有方法都快。特别地，在 CIFAR-10 上生成一张图像时，它比 softmax Transformer 快了约 6.6 倍。注意，我们的线性自回归 Transformer 是唯一一个在每种情况下都在 CPU 上比 GPU 上更快的方法。这是因为将注意力作为 RNN 计算的成本如此之低，以至于主要的计算瓶颈变成了对序列的不可避免的外层循环。

6 Conclusions（结论）

原文翻译

在本工作中，我们提出了线性 Transformer，该模型显著降低了原始 Transformer 的内存和计算成本。特别地，通过利用矩阵乘积的结合律性质，我们能够将自注意力的时间和内存相对于序列长度线性增长。我们表明，我们的模型可以与因果掩码一起使用，并仍然保持其线性渐近复杂度。最后，我们将 Transformer 模型表示为循环神经网络，这使得我们能够在自回归任务上执行快数千倍的推理。
这一性质为未来研究开辟了多个方向，涉及 RNN 和 Transformer 中信息的存储与检索。另一个值得探索的研究方向与线性注意力的特征映射选择有关。例如，用随机傅里叶特征来近似 RBF 核可以允许我们利用已用 softmax 注意力预训练的模型。

全文总结：一张图看懂这篇论文

核心问题： Transformer 的自注意力是 $\mathcal{O}(N^2)$，太慢了。
核心方法： 三步走——

1. **核函数分解：** 把 $\mathrm{sim}(Q_i, K_j)$ 写成 $\phi(Q_i)^T \phi(K_j)$
2. **利用结合律：** 把 $(\phi(Q)\,\phi(K)^T)V$ 变成 $\phi(Q)\,(\phi(K)^T V)$，避免 $N \times N$ 矩阵
3. **写成 RNN：** 因果掩码下，用 $S_i = S_{i-1} + \phi(K_i)\,V_i^T$ 递推，每步常数时间

核心结果：

- 训练复杂度：$\mathcal{O}(N^2) \to \mathcal{O}(N)$
- 推理复杂度：$\mathcal{O}(N^2)$ 总量 $\to \mathcal{O}(1)$ 每步

- 实际加速：最高 **4000 倍**
- 性能损失：很小（在多数任务上接近标准 Transformer）

核心洞察：任何带因果掩码的 Transformer 都可以看作一个 RNN。
未来方向：

- 更好地理解 Transformer-RNN 中信息存储与检索的机制
- 探索更好的特征映射 ϕ（如用随机傅里叶特征近似 RBF 核，从而可以复用已有的 softmax 预训练模型）

A 附录 A：线性因果注意力的前向和反向传播伪代码

Algorithm 1: 线性 Transformer 因果掩码的前向和反向传播

```
function forward(φ(Q), φ(K), V):
    V̄ ← 0, S ← 0                                   // S 是 C × M 的累加矩阵
    for i = 1, ..., N do
        S ← S + φ(K_i) V_i^T                        // 更新记忆（公式17）
        V̄_i ← φ(Q_i)^T S                            // 查询记忆得到输出
    end
    return V̄
end

function backward(φ(Q), φ(K), V, G):
    /* G = ∇_V̄ L：损失对前向输出的梯度                                  */
    S ← 0, ∇_φ(Q) L ← 0
    for i = 1, ..., N do                            // 正向扫描，计算 ∇_Q
        S ← S + φ(K_i) V_i^T
        ∇_φ(Q_i) L ← G_i · S^T
    end
    S ← 0, ∇_φ(K) L ← 0, ∇_V L ← 0
    for i = N, ..., 1 do                            // 反向扫描，计算 ∇_K, ∇_V
        S ← S + φ(Q_i) G_i^T
        ∇_V_i L ← S^T φ(K_i)
        ∇_φ(K_i) L ← S V_i
    end
    return ∇_φ(Q) L, ∇_φ(K) L, ∇_V L
end
```

算法解读

前向传播（forward）：从 $i=1$ 到 N 正向扫描，维护一个 $C \times M$ 的累加矩阵 S。每步加上新的 $\phi(K_i)\,V_i^T$，然后用 $\phi(Q_i)$ 查询得到输出。
反向传播（backward）：

- 计算 ∇_Q：再次正向扫描（因为 ∇_Q 的公式中是 $\sum_{j=1}^{i}$，正向累加）
- 计算 ∇_K, ∇_V：反向扫描（因为公式中是 $\sum_{j=i}^{N}$，反向累加）

整个过程只需要维护一个 $C \times M$ 大小的累加矩阵——**内存为常数**！不需要存储所有中间的 S_i。时间 $\mathcal{O}(NCM)$，即线性。

B 附录 B：完整参考文献一览表

下表汇总了本论文引用的全部 39 篇文献，按在论文中出现的先后顺序排列。每篇文献给出作者/年份、简要内容和在本文中的引用原因。

文献	主要内容	在本文中的引用原因
Vaswani et al. (2017) “Attention is All You Need”	提出 Transformer 架构，用自注意力完全替代了 RNN 和 CNN	本文的基础模型。本文所有工作都是在改进 Transformer 的自注意力机制
Sutskever et al. (2014) Seq2Seq	提出编码器-解码器框架的序列到序列学习	作为神经机器翻译的背景引用，说明 Transformer 最初的应用场景
Bahdanau et al. (2015) Attention for NMT	为序列到序列模型引入注意力机制	同上，说明注意力机制在 NMT 中的起源
Devlin et al. (2019) BERT	通过双向掩码语言建模预训练的 Transformer	说明 Transformer 在 NLP 中的巨大成功（双重引用：作为 NLP 成功案例 + 掩码预训练代表）
Sperber et al. (2018) Self-Attention for Speech	将自注意力应用于语音处理	说明 Transformer 在音频领域的应用
Parmar et al. (2019) Stand-Alone Self-Attention	将自注意力作为独立模块用于视觉模型	说明 Transformer 在图像领域的应用
Radford et al. (2018) GPT-1	通过自回归预训练学习语言表示	作为自回归预训练方法的代表
Radford et al. (2019) GPT-2	展示大规模语言模型的零样本学习能力	同上；同时在图像生成部分引用以说明 Transformer 的生成能力
Yang et al. (2019) XLNet	广义自回归预训练，结合了自回归和双向上下文	作为预训练方法的补充引用
Song et al. (2019) MASS	掩码序列到序列预训练	同上
Liu et al. (2020) RoBERTa	对 BERT 预训练策略进行鲁棒性优化	同上
Dai et al. (2019) Transformer-XL	通过段级递归实现超长上下文的语言建模	说明” 增加上下文长度” 的已有尝试，但渐近复杂度未改善
Child et al. (2019) Sparse Transformer	对注意力矩阵进行稀疏分解，复杂度降至 $O(N\sqrt{N})$	本文的前导工作之一。降低了复杂度但不加速推理
Kitaev et al. (2020) Reformer	用局部敏感哈希将复杂度降至 $O(N \log N)$	本文最重要的对比基线。在所有实验中都作为比较对象
Michel et al. (2019) “Are 16 Heads Better”	研究注意力头剪枝，发现很多头可以移除	作为” 权重剪枝” 提高效率这一思路的代表

文献	主要内容	在本文中的引用原因
Lan et al. (2020) ALBERT	跨层参数共享和嵌入分解以减少参数	作为”权重分解”提高效率这一思路的代表
Zafrir et al. (2019) Q8BERT	将 BERT 量化为 8 位整数	作为”权重量化”提高效率这一思路的代表
Clark et al. (2020) ELECTRA	提出替换词检测预训练目标，提高样本效率	作为更高效预训练方法的代表
Lample et al. (2019) Product-Key Attention	用乘积键注意力以低开销增加模型容量	说明在不改变渐近复杂度的情况下提高效率的方法
Sukhbaatar et al. (2019) Adaptive Attention Span	让每个注意力头学习最优的注意力跨度	作为”上下文扩展”方法的对比——渐近复杂度未变
Tsai et al. (2019) Transformer Dissection	从核函数视角理解 Transformer 注意力	核心理论基础。本文的线性化思路直接受此启发
Cordonnier et al. (2020) Self-Attention = Conv	证明多头自注意力可表达任何卷积层	作为”理解自注意力”研究方向的另一角度。与本文”Transformer=RNN”形成呼应
Goodman (2001) Class-based Softmax	基于类别的层次化 softmax 加速	说明 softmax 计算瓶颈是一个长期存在的问题
Morin & Bengio (2005) Hierarchical Softmax	层次化 softmax 用于语言模型	同上
Mnih & Hinton (2009) Scalable Hierarchical LM	可扩展的层次化分布式语言模型	同上
Blanc & Rendle (2017) Adaptive Sampled Softmax	用自适应采样近似 softmax	用特征映射线性点积近似 softmax 的思路直接启发了本文
Rawat et al. (2019) Sampled Softmax + RFF	用随机傅里叶特征进行采样 softmax	同上
Shen et al. (2020) Efficient Attention	将线性化注意力用于图像目标检测	与本文同期的工作。本文在此基础上进一步发展了自回归模型和 RNN 等价性

文献	主要内容	在本文中的引用原因
Dehghani et al. (2018) Universal Transformer	在 Transformer 中引入深度维度上的递归	用于区分——本文的”递归”是关于时间的，而非深度的
Hochreiter & Schmidhuber (1997) LSTM	提出长短期记忆网络	经典 RNN 代表。本文揭示了 Transformer 与 RNN 家族的联系；语音实验中用作基线
Clevert et al. (2015) ELU	提出指数线性单元激活函数	本文的核心特征映射 $\phi(x) = \mathrm{elu}(x) + 1$ 就基于 ELU
Salimans et al. (2017) PixelCNN++	引入混合 logistics 分布建模像素值	本文的图像生成实验使用了这一输出建模方式
LeCun et al. (2010) MNIST	手写数字识别基准数据集	本文的图像生成实验数据集之一
Krizhevsky et al. (2009) CIFAR-10	10 类自然图像分类基准数据集	本文的图像生成实验数据集之一
Liu et al. (2019) RAdam	带方差修正的 Adam 优化器	本文所有实验（除 LSTM 基线外）使用的优化器
Paul & Baker (1992) WSJ Dataset	Wall Street Journal 语音语料库设计	语音识别实验使用的数据集
Graves et al. (2006) CTC Loss	连接时序分类损失函数	语音识别实验使用的训练损失
Kingma & Ba (2014) Adam	提出 Adam 优化器	语音识别实验中 LSTM 基线使用的优化器
Paszke et al. (2019) PyTorch	PyTorch 深度学习框架	本文所有实验的实现框架

致谢

原文翻译

Angelos Katharopoulos 受瑞士国家科学基金会资助，资助编号 FNS-30209 “ISUL” 和 FNS-30224 “CORTI”。Apoorv Vyas 受瑞士国家科学基金会资助，资助编号 FNS-30213 “SHISSM”。Nikolaos Pappas 受瑞士国家科学基金会资助，资助编号 P400P2_183911 “UNISON”。